IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

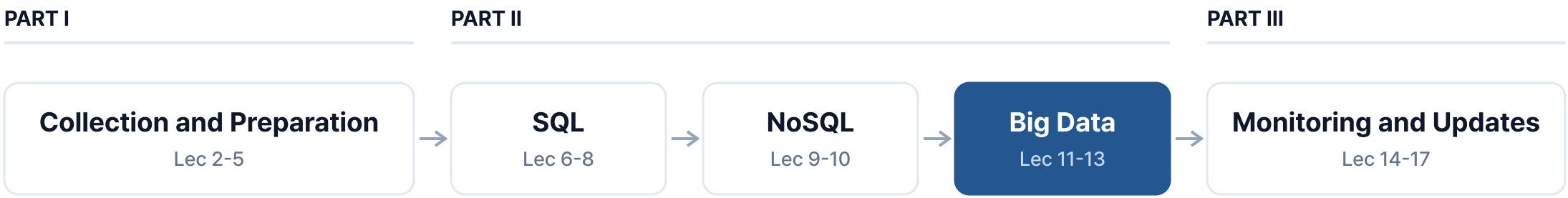
12.

Big Data II: Data Warehousing

Sangwoo Mo · POSTECH

Industrial & Management Engineering

Where We Are


PART I
PART II
PART III
Collection and Preparation
Lec 2-5
SQL
Lec 6-8
NoSQL
Lec 9-10
Big Data
Lec 11-13
Monitoring and Updates
Lec 14-17

Recap: Distributed Batch Processing

A distributed program

```
(sales.groupBy("product_id")
      .sum("amount")
      .show())
```

A distributed program still needs infrastructure, storage and a way to run again

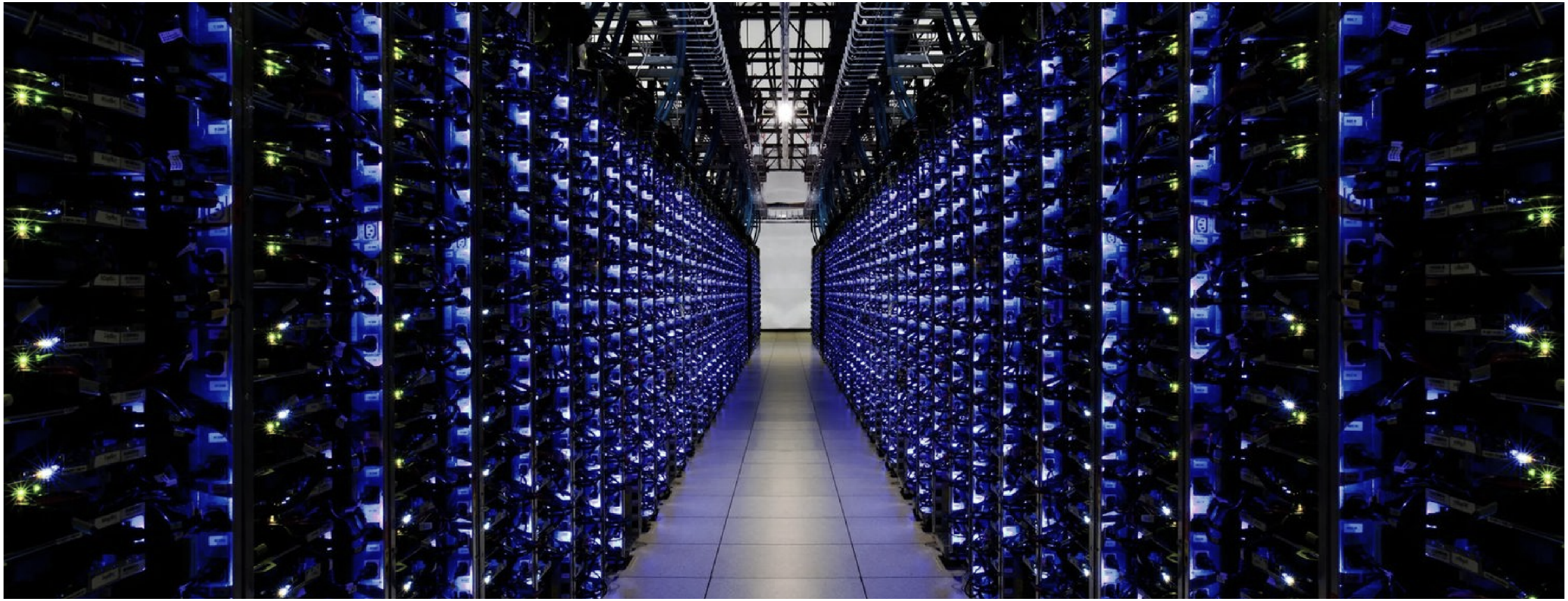
Outline: Big Data II

01	**Cloud and Serverless**	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

01 · Cloud and Serverless

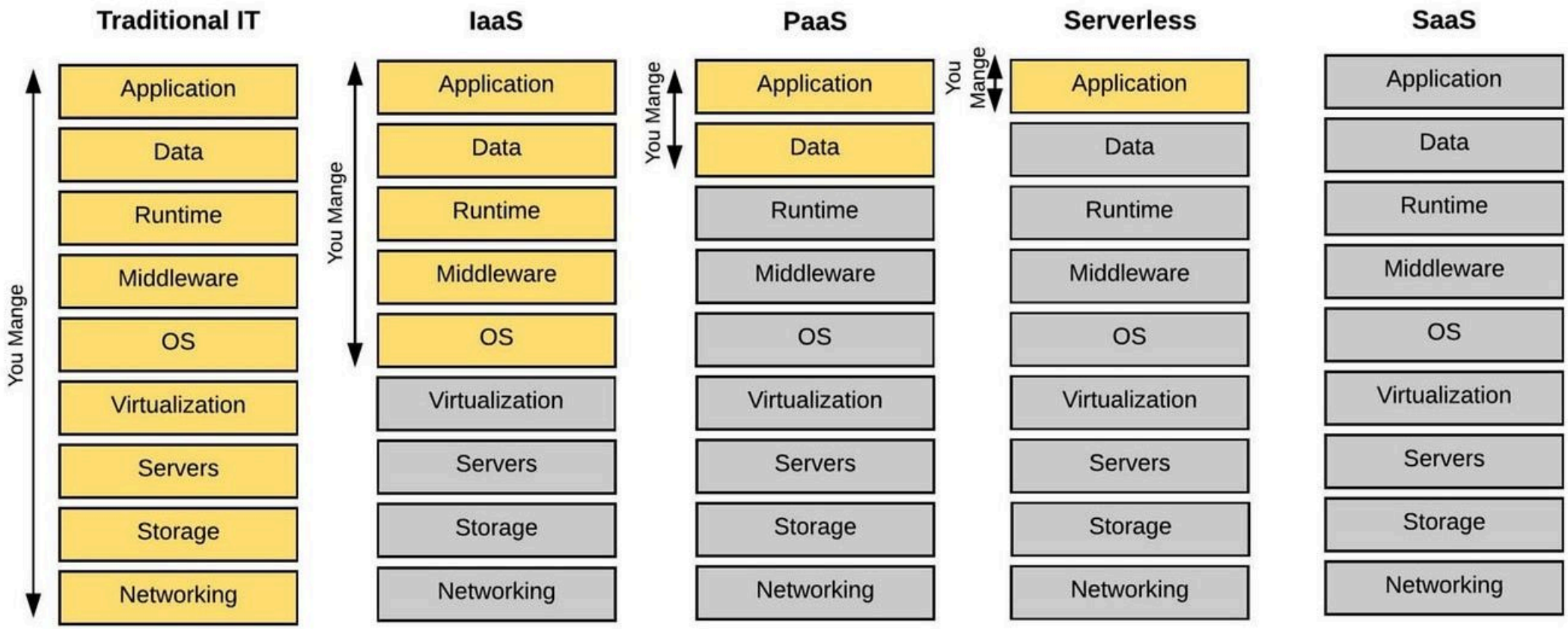
01	**Cloud and Serverless**	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

Why Managed Infrastructure



The cloud still runs on physical servers that somebody must operate

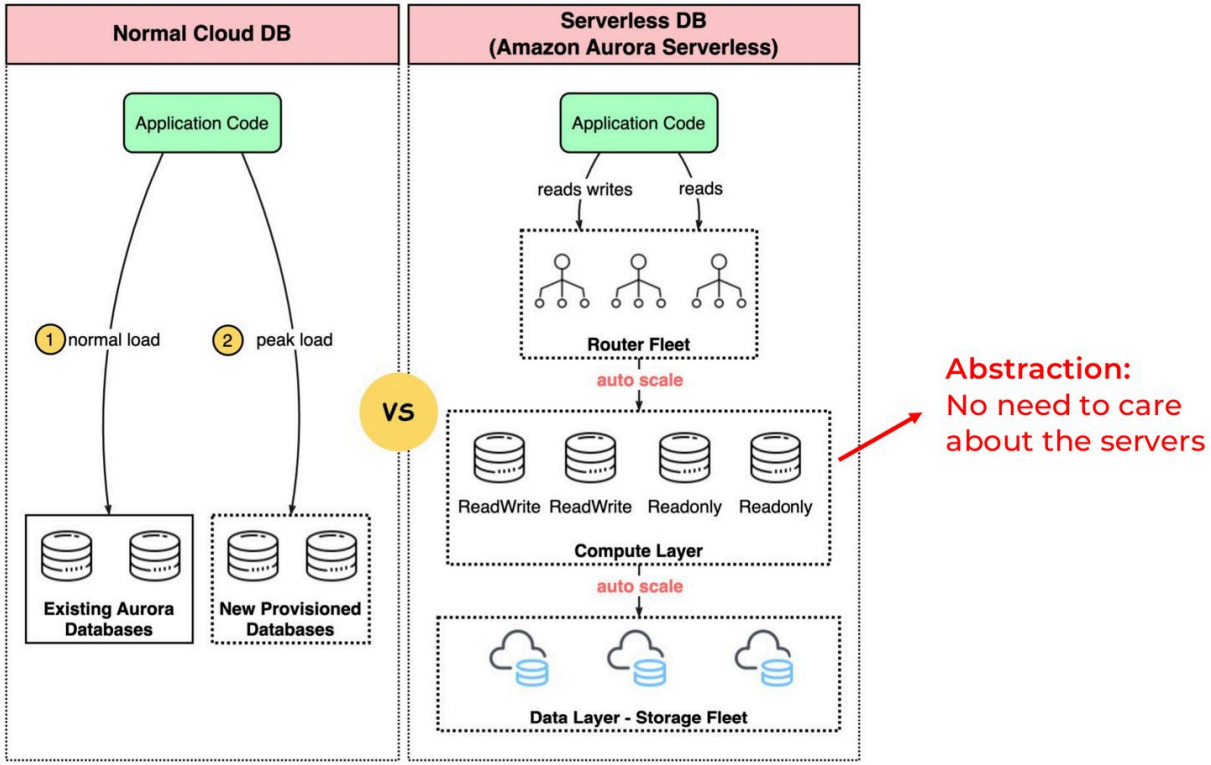
Cloud Service Models



Moving right transfers more infrastructure operations to the provider

Serverless Computing

Serverless transfers server provisioning and operation to a managed service



The service scales servers behind the application interface

Spark and BigQuery

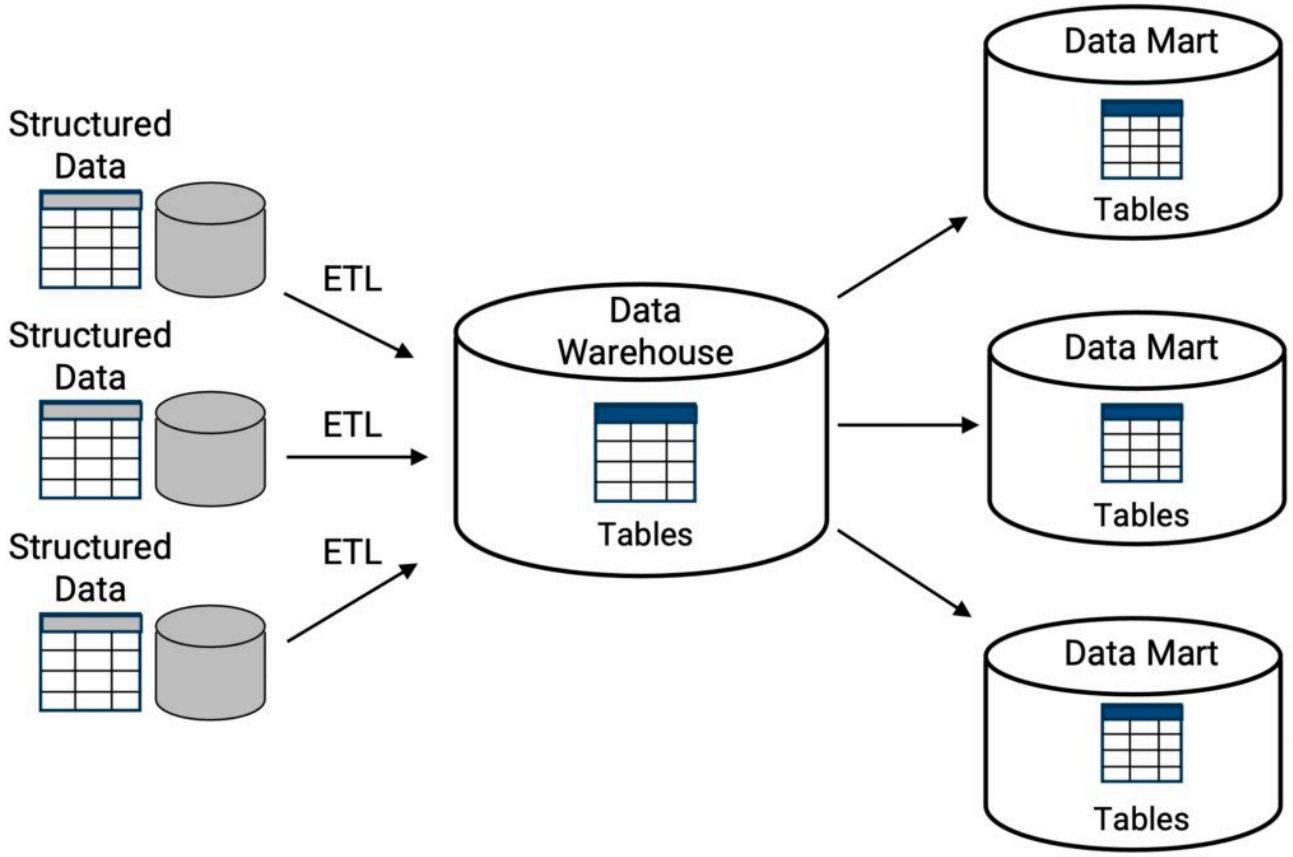
Tool	User expresses	Typical responsibility
Spark	A distributed program	Choose transformations and execution environment
Managed Spark service	A Spark program	Use a provider-managed execution environment
BigQuery	An analytical SQL query	Define tables and the analytical question

An execution engine and a managed analytical service solve different parts of the workload

02 · Analytical Warehouses

01	Cloud and Serverless	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	Lakes and Lakehouses	Where do files become tables?
04	Data Orchestration	Which task runs after which, and safely again?

Why an Analytical Warehouse



Shared analytical tables connect multiple sources to consistent departmental reports

A Query Has a Data Contract

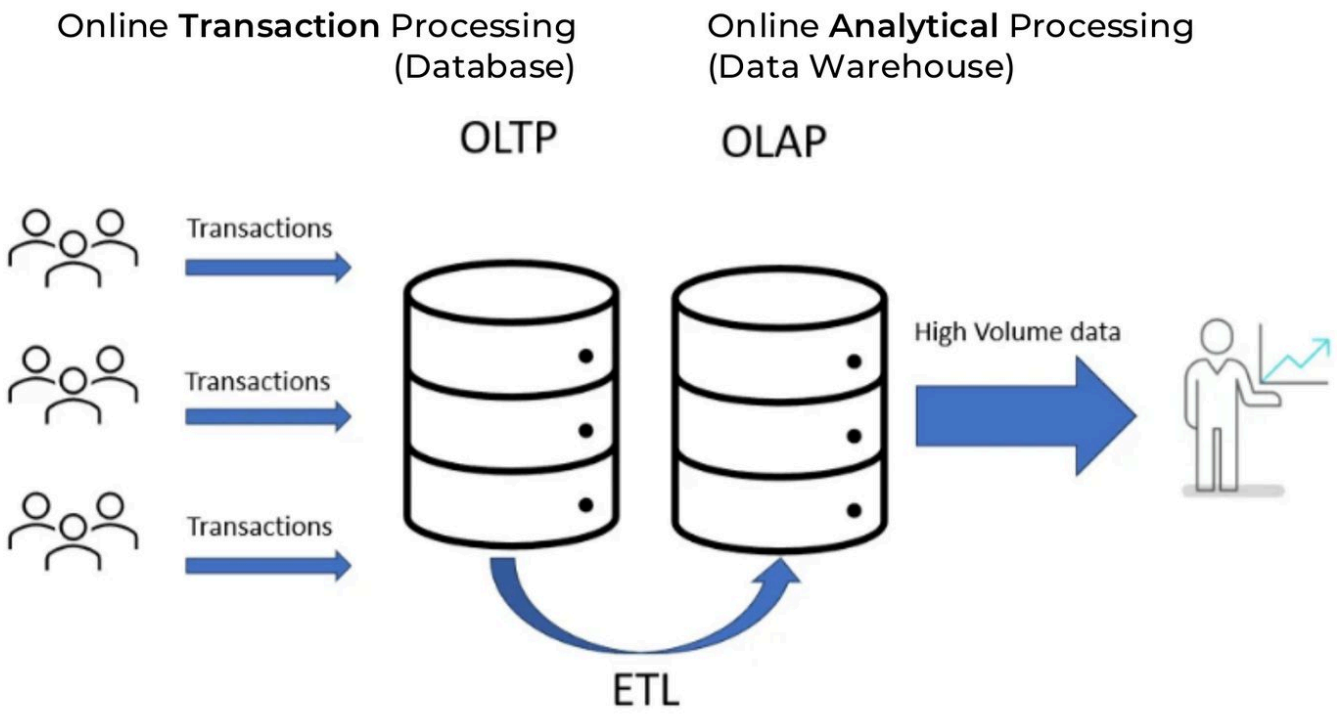
Grain means what one row represents; here, one sale event

Sale	Day	Product	Amount
1	Oct 2	P1	300
2	Oct 2	P2	200
3	Oct 3	P1	400
4	Oct 3	P2	100

Four sale rows produce two daily rows: 500 and 500

OLTP and OLAP

Online Transaction Processing (OLTP) and Online Analytical Processing (OLAP)



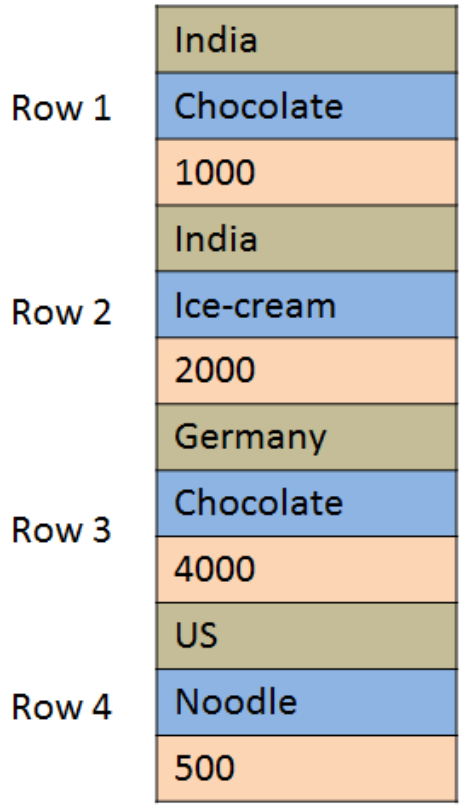
Small transactional updates and large analytical scans place different demands on storage

The Same Values on Disk

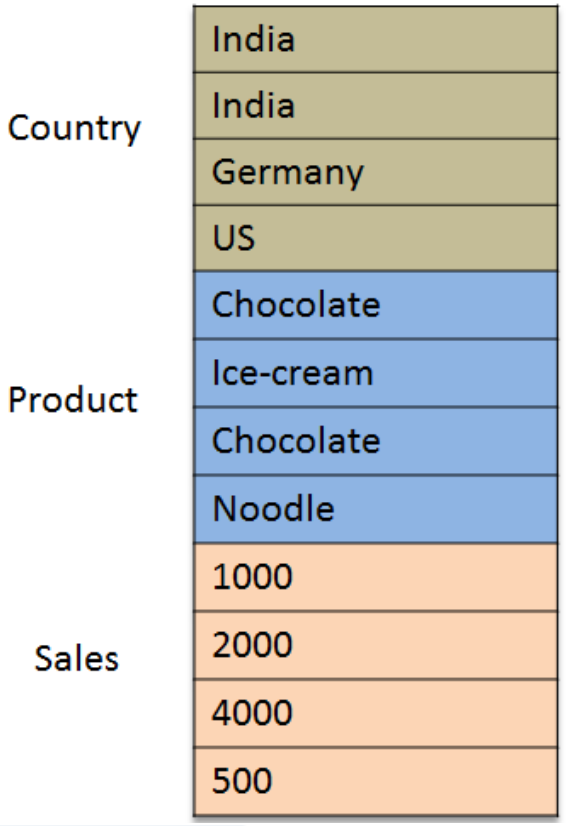
Logical table

Country	Product	Sales
India	Chocolate	1000
India	Ice-cream	2000
Germany	Chocolate	4000
US	Noodle	500

Row store



Column store



Twelve values retain their meaning while their physical order changes

Read Cost Before Execution

Model	Values read	Why
Row scan	N × C	Read complete rows
Column scan	N × K	Read output AND filter columns
Partition + projection	f × N × K	Skip unrelated date ranges first

`N = rows` `C = all columns` `K = required columns` `f = selected fraction`

Read fewer columns and fewer partitions; both reduce input without changing the answer

Demo: Storage Access

Predict the required columns and date ranges before reading

Query: October: SUM(amount) | Date partitions | Read next group | Finish scan | Reset

Row layout · 8 values per row

Row 1	1	09/30	P1	North	USD	1	300	300
Row 2	2	10/02	P2	South	USD	2	100	200
Row 3	3	10/03	P1	North	USD	2	200	400
Row 4	4	10/03	P2	South	USD	1	100	100
Row 5	5	10/04	P1	North	USD	1	250	250
Row 6	6	10/04	P2	South	USD	1	150	150

0 values × 8 = 0 bytes read

Column layout · 6 values per column

date	09/30	10/02	10/03	10/03	10/04	10/04
amount	300	200	400	100	250	150

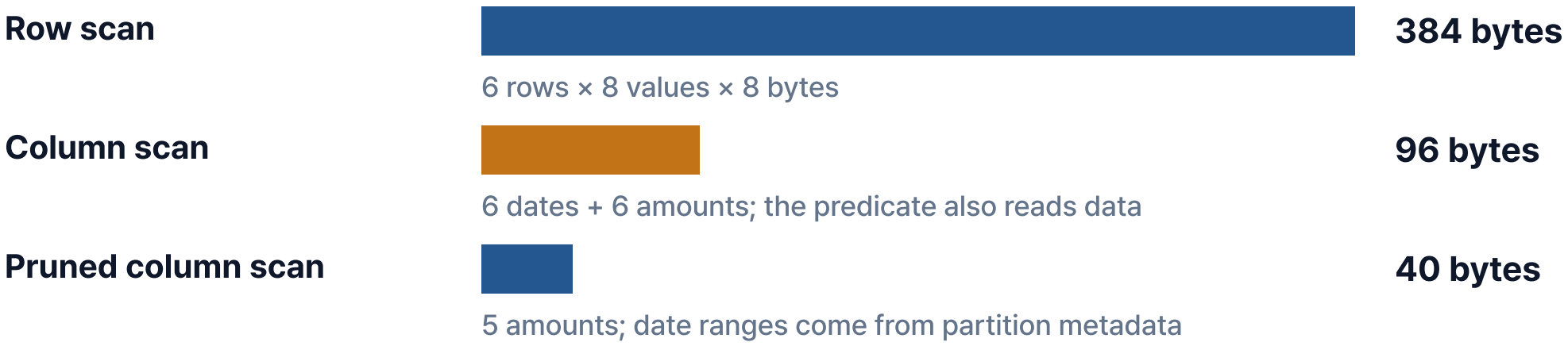
0 values × 8 = 0 bytes read

Predict bytes, then read; count values rather than differently sized groups

Model: 8 bytes/value; full groups; no indexes or compression. Date metadata enables pruning.

Answer: Projection and Partitions

Teaching model: 6 rows, 8 equal-width values, no compression or indexes



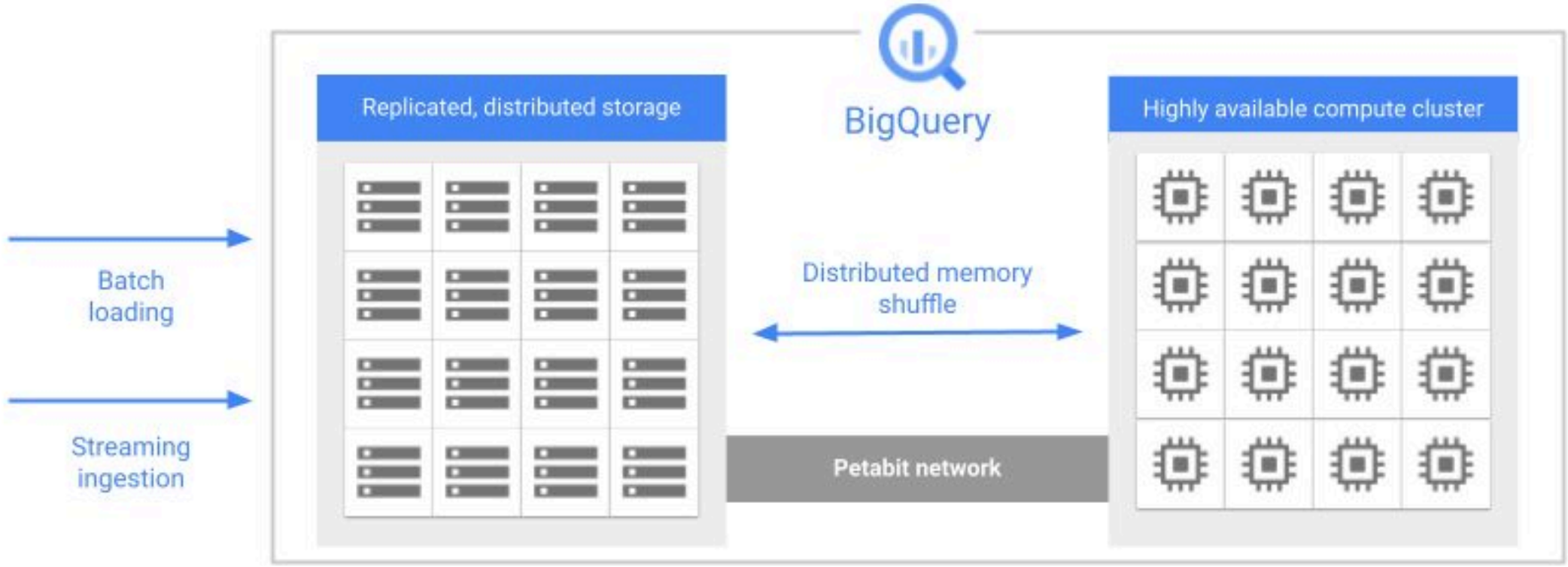
October total stays 1100; the work changes rather than the result

Columns Enable Compact Encoding

Order of product IDs	Run-length encoding	Runs
0, 1, 0, 1, 0, 1	(0,1), (1,1), (0,1), (1,1), (0,1), (1,1)	6
0, 0, 0, 1, 1, 1	(0,3), (1,3)	2

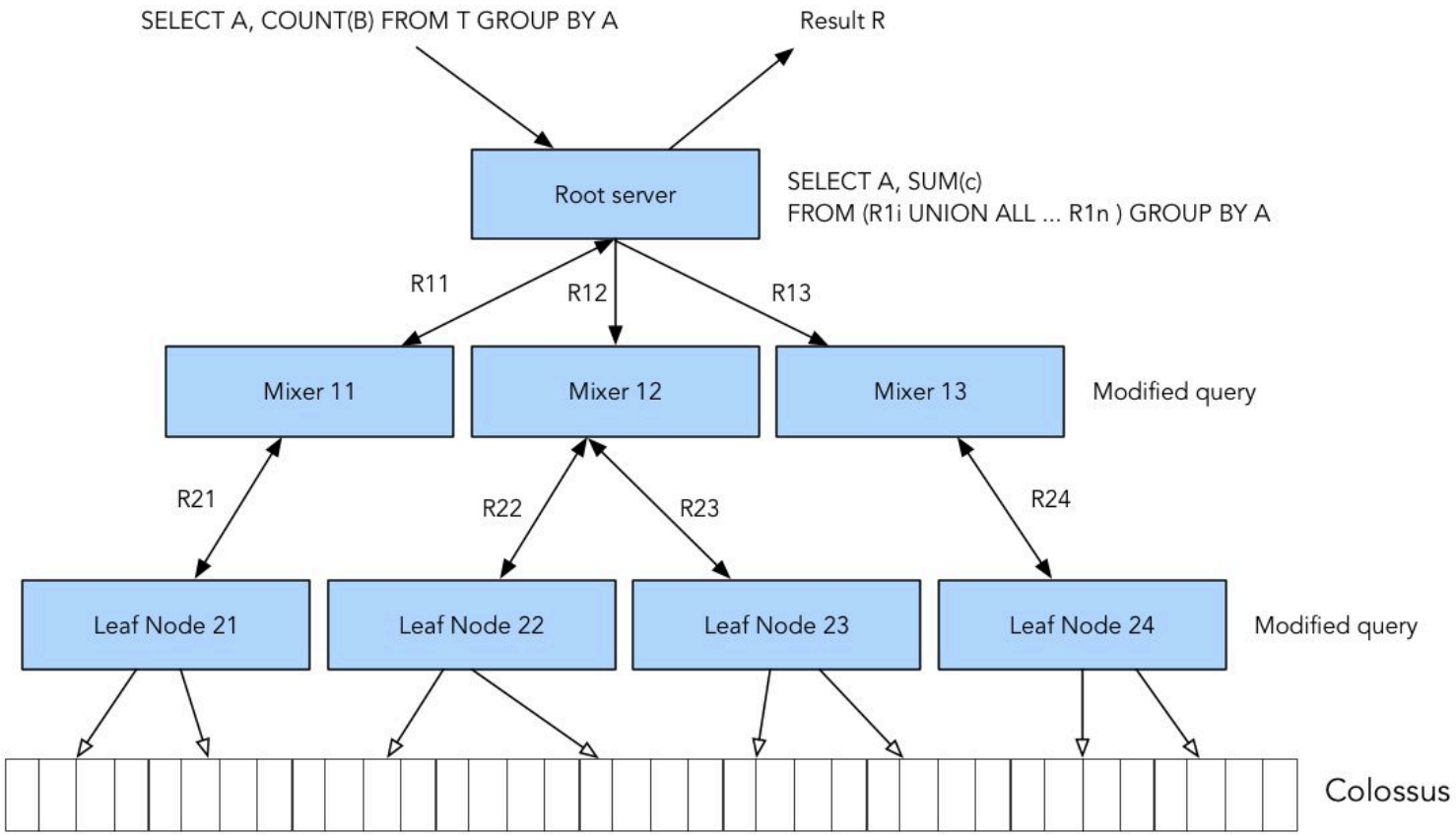
Equal values can compress differently because ordering changes the number of runs

BigQuery Storage and Compute



Persistent table storage and query execution are managed as separate layers

Distributed Query Execution



Workers read portions of the table and combine partial results

Partial Results Must Be Mergeable

Count rows locally, then merge counts by country

Worker	Partial counts	Root merge
A	India: 2, US: 1	India: 2 + 1 = 3
B	India: 1, US: 3	US: 1 + 3 = 4

Local COUNT results become a global SUM over the same group key

Averages Need Sums and Counts

Worker	Values	Partial state
A	10, 10	sum = 20, count = 2
B	90	sum = 90, count = 1

```
AVG = (20 + 90) / (2 + 1) = 36.67
(10 + 90) / 2 = 50    -- wrong: unequal group sizes
```

An aggregate is distributed by preserving the state needed for a correct merge

Distinct Counts Need More State

COUNT can merge scalars; exact DISTINCT must account for overlap

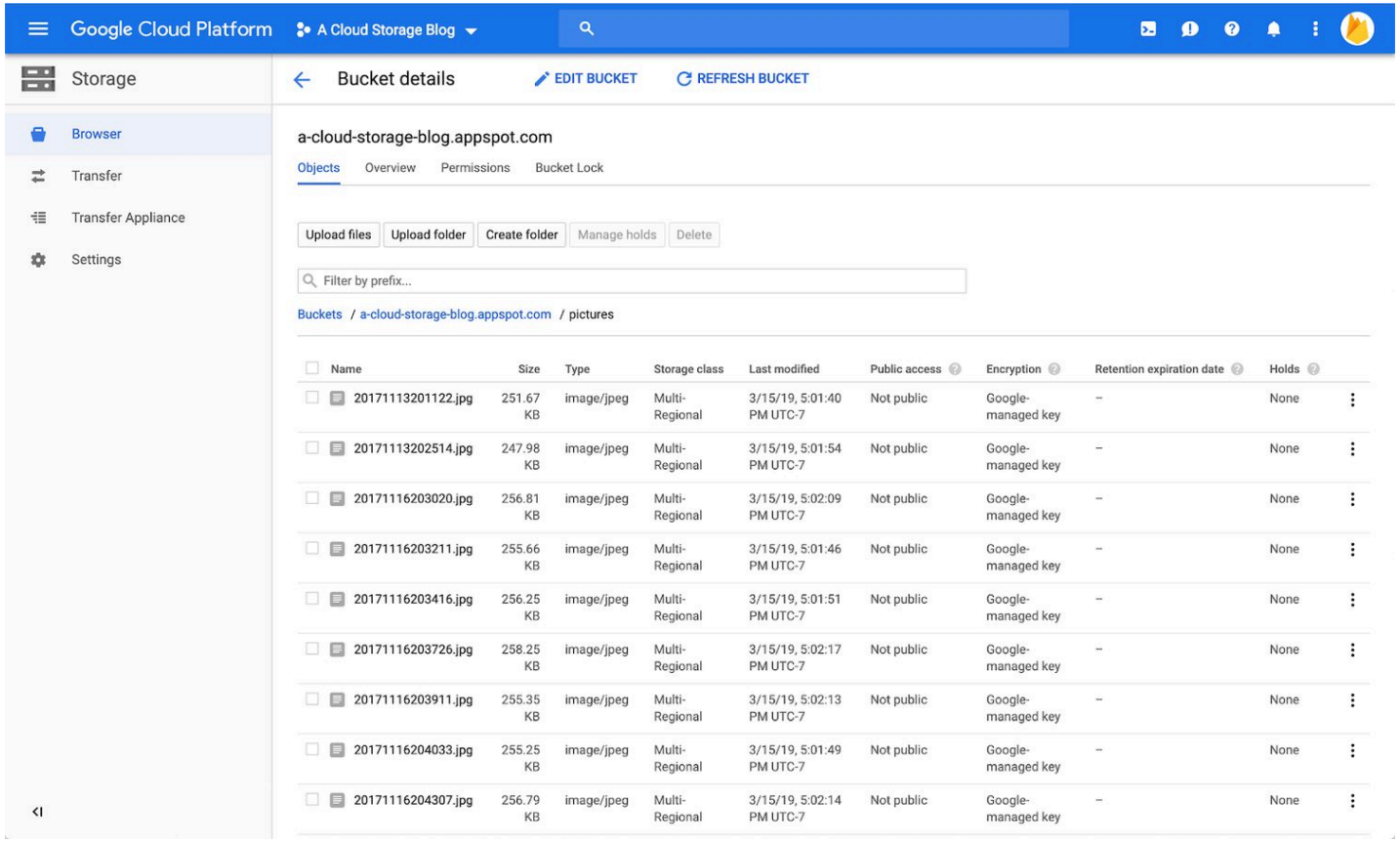
Worker	Product IDs	Local distinct count
A	P1, P2	2
B	P1, P3	2
Merge	P1, P2, P3	3, rather than 2 + 2

A correct distributed aggregate must preserve enough information to merge overlapping inputs

03 · Lakes and Lakehouses

01	Cloud and Serverless	Who operates the infrastructure?
02	Analytical Warehouses	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	Data Orchestration	Which task runs after which, and safely again?

Why Keep a Data Lake



Source files can remain in object storage before analytical tables are defined

Keep the Grain You May Need

Grain determines which future questions remain answerable

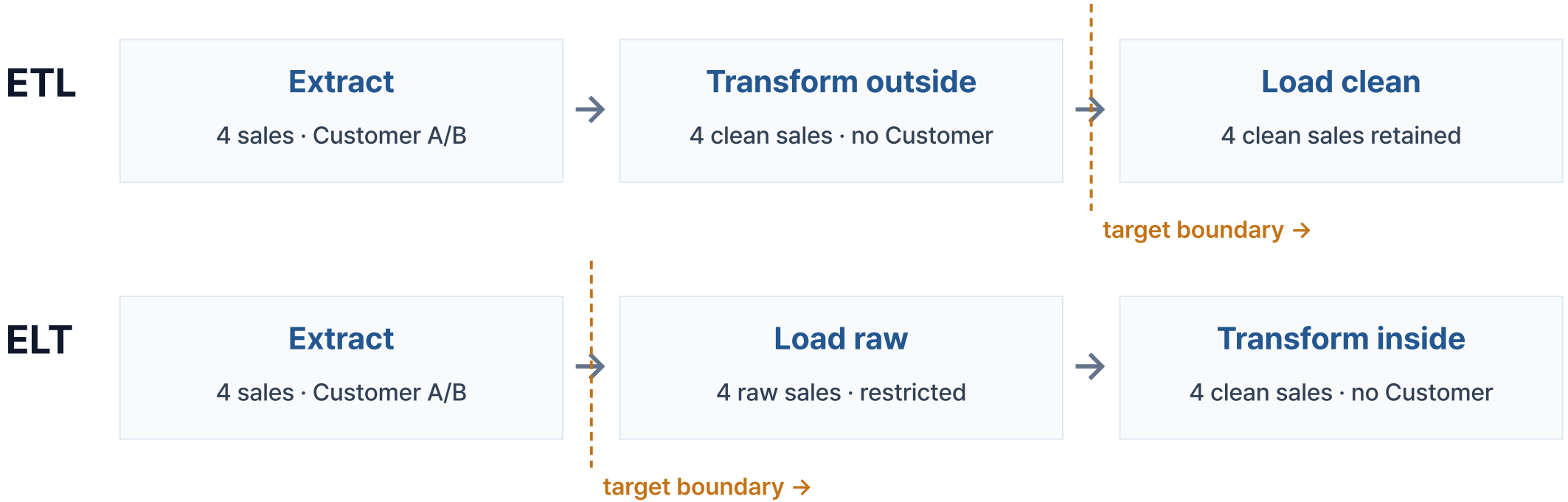


Daily totals: 500 / 500 in BOTH cases; the product allocation is not recoverable

Two different product allocations produce identical daily totals

ETL and ELT

Extract, Transform, Load (ETL) or Extract, Load, Transform (ELT)



The same four sale rows cross the target boundary at different transformation stages

Choosing ETL or ELT

Constraint	Reasonable placement	Separate decision
Raw identifiers cannot enter target	Transform outside: ETL	Archive source in an allowed location
Restricted landing area + warehouse compute	Transform inside: ELT	Control access to raw data
New analytical questions will arrive	Either placement can work	Retain reusable detail or a replayable source

Data boundaries and available compute choose location; retention determines what can be rebuilt

Demo: ETL and ELT

Hold grain fixed, then change retention as an independent policy

Report By day ☑ Retain reusable detail in BOTH paths Next stage Replay source Reset

ETL

Same source · 4 sales

Day	Product	Amount	Customer
10/02	P1	300	A
10/02	P2	200	B
10/03	P1	400	A
10/03	P2	100	B

→ **Transform outside · remove Customer**

Not started

→ **Load clean detail · report**

Not ready

ELT

Same source · 4 sales

Day	Product	Amount	Customer
10/02	P1	300	A
10/02	P2	200	B
10/03	P1	400	A
10/03	P2	100	B

→ **Load raw · restricted area**

Not started

→ **Transform inside · report**

Not ready

Same transformation and grain; predict which boundary contains Customer

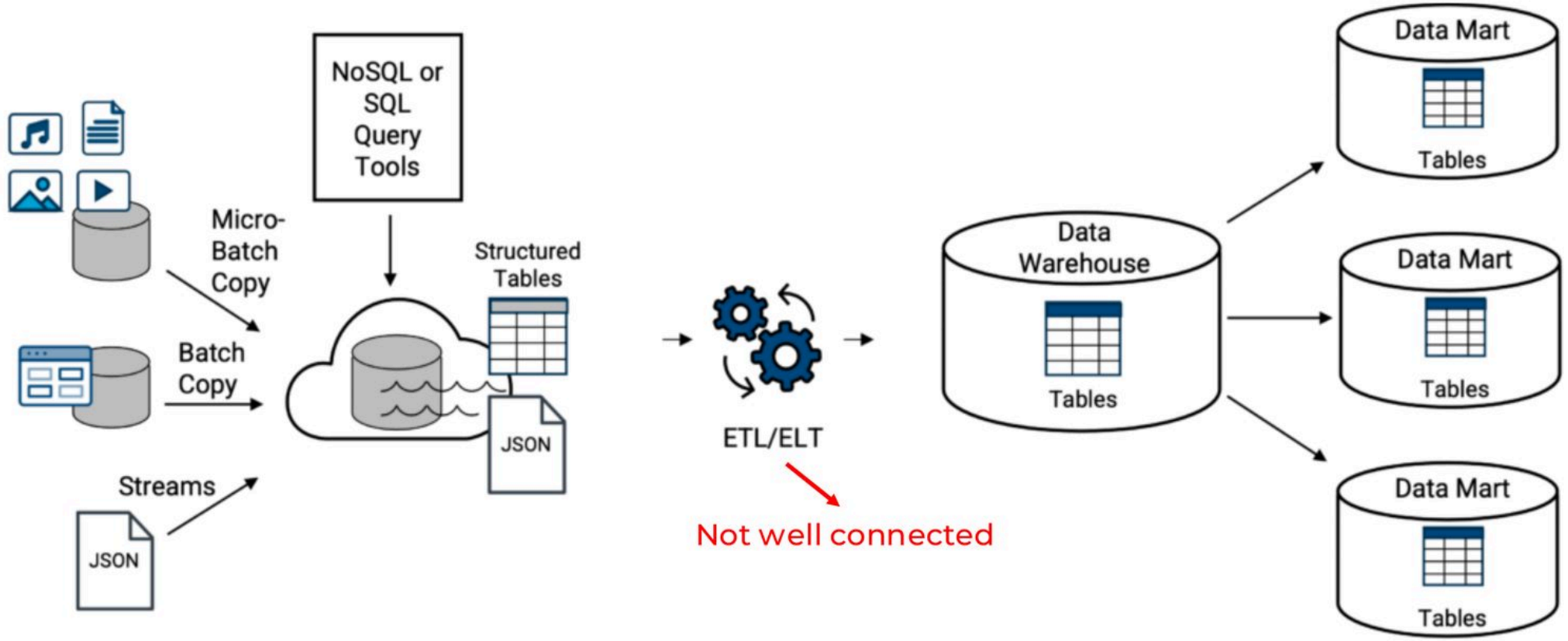
Same transformation: remove Customer; keep sale-level detail. Retention is a separate policy from transformation location.

Answer: Location Is Not Retention

Policy in BOTH paths	By day	Switch to product
Retain reusable sale detail	500 / 500	Rebuild: P1 700; P2 300
Keep daily report only	500 / 500	Replay source in BOTH paths
No detail and no replayable source	500 / 500	Product allocation cannot be recovered

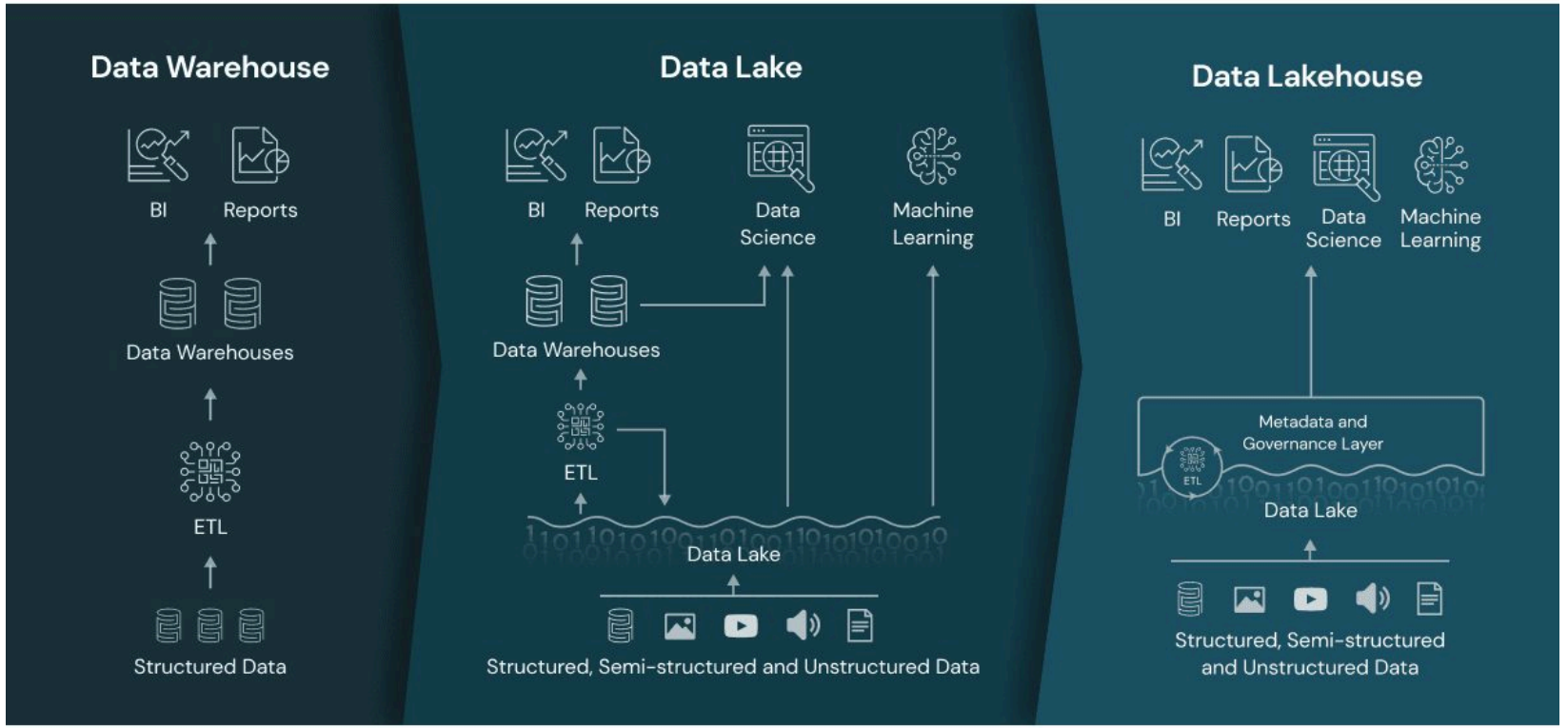
Reprocessing depends on retained information, not the order of three letters

From Lake to Warehouse



Separate raw and analytical stores introduce copying, transformations and refresh dependencies

Why a Lakehouse



Lakehouse adds table management over lake files for shared analytical use

What Makes a Lakehouse Work

Layer	Mechanism	Reason
Table management	Transactions and versioned metadata	Readers see a consistent table
Query execution	Columnar files, pruning and statistics	SQL scans avoid unrelated data
Shared access	SQL and DS/ML use the same snapshot	Fewer independent data copies

Table transactions, query execution and shared data access turn files into managed tables

Commit a Table Snapshot

Object completion

Upload new-A → complete object

No partial object is visible

Directory: old-A + old-B + new-A

Complete files can still form a wrong table

Table publication

Snapshot 1: old-A + old-B

Prepare: new-A + old-B

Atomic commit → Snapshot 2

Reader: 550 + 500 = 1050

Atomic object writes do not provide an atomic table-wide replacement

Demo: Table Publication

Stop after object completion but before table publication

Next write stage | Stop writer now | Reset | ☑ Publish through a snapshot

Complete objects in storage

File	Day	Sales
old-A	Oct 2	500
old-B	Oct 3	500

Every listed object is complete

Published file list

Snapshot 1

One complete table version



Reader result

File	Sales
old-A	500
old-B	500

Total: 1000

Initial table: 500 + 500 = 1000

Answer: Consistent Does Not Mean Correct

Guarantee	What it establishes	What remains to check
Object atomicity	A whole object is visible	Which files form the table?
Table snapshot	One committed file list	Were its rows and totals correct?
Workflow dependencies	Required upstream tasks completed	Did they use the right interval and keys?

A complete snapshot can contain wrong data; publication still needs explicit data checks

Tools Around a Lakehouse

Compute and platform	**Spark, Databricks**	run transformations and queries
Table format	**Iceberg, Delta Lake**	manage table metadata and changes
Columnar file format	**Parquet**	encode values in files
Object storage	**S3, Cloud Storage**	store the data files

Storage, file format, table format and compute belong to different layers

04 · Data Orchestration

01	Cloud and Serverless	Who operates the infrastructure?
02	Analytical Warehouses	How are many rows read and summarized?
03	Lakes and Lakehouses	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

Why Data Orchestration

Task can finish successfully	Yet the result can be wrong	Required contract
Load orders	Wrong date loaded	Fixed input interval
Join products	Duplicate product keys multiply sales	One matching product per sale
Publish table	A complete but incorrect snapshot	Validate before publishing

A workflow coordinates readiness, computation, validation and publication

Airflow at Airbnb

2014
started at Airbnb by M. Beauchemin

2015
announced as open source

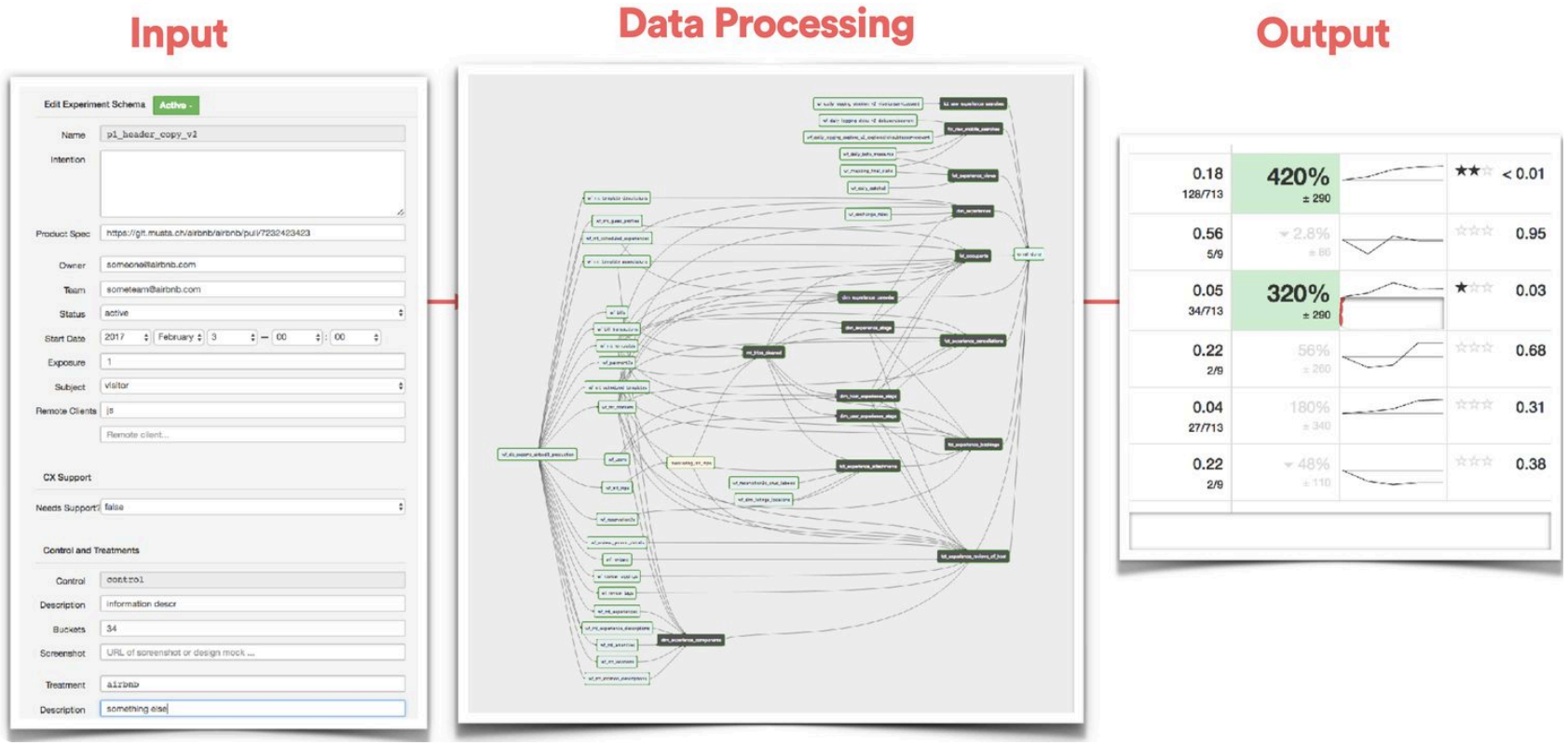
2016
Apache incubation

2019
Apache top-level project

Airflow grew from a company workflow tool into an Apache project

Case study: Airbnb Experiment Analytics

Historical Airbnb workflow: configuration, computation and experiment metrics



Experiment configuration drives a dependency graph that produces analytical results

A Sales Workflow DAG

Tasks have input and output contracts, as well as dependencies

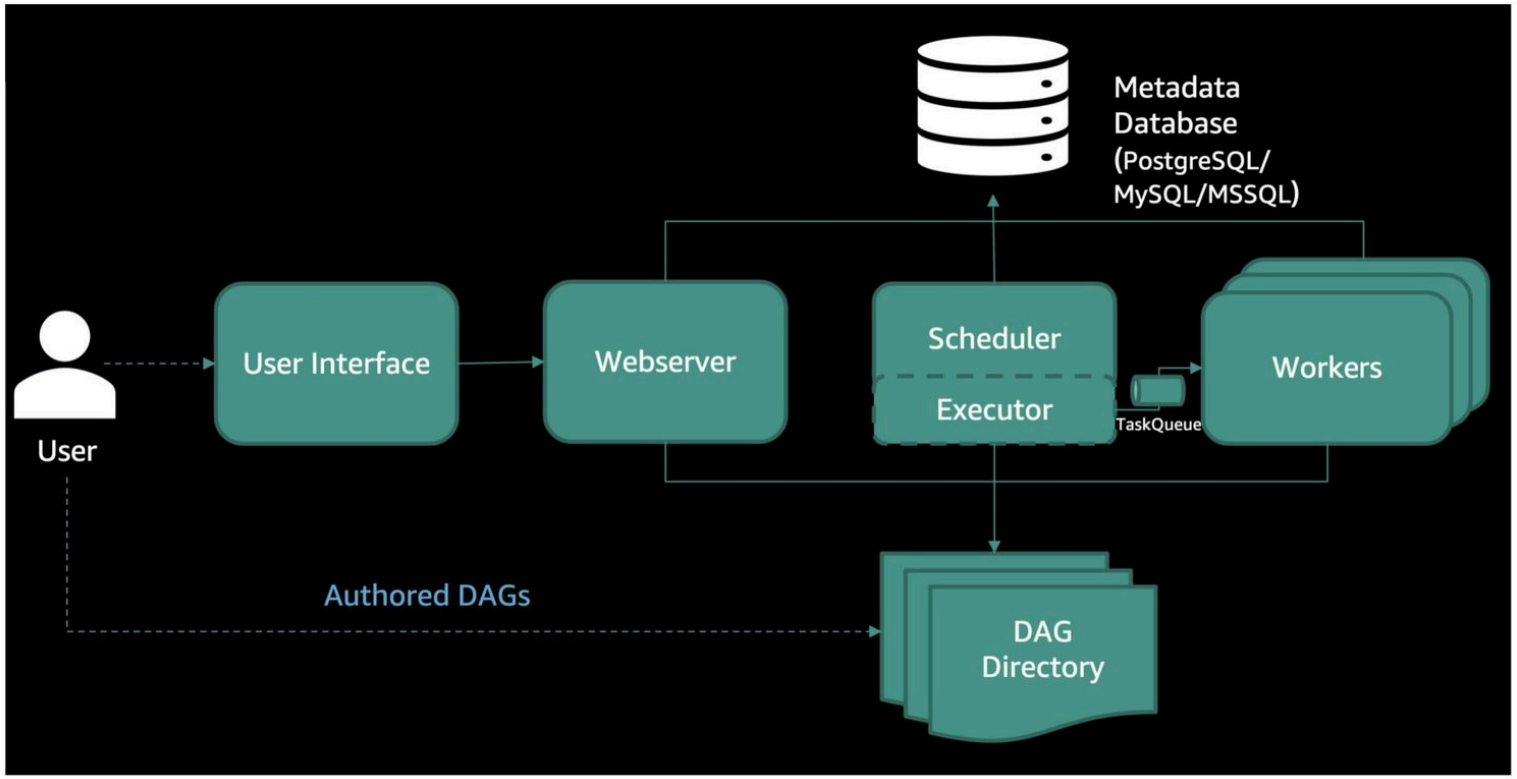


An edge requires the upstream output; it does not move the sales rows

The join preserves four sales; validation is upstream of publication

Airflow Control and Workers

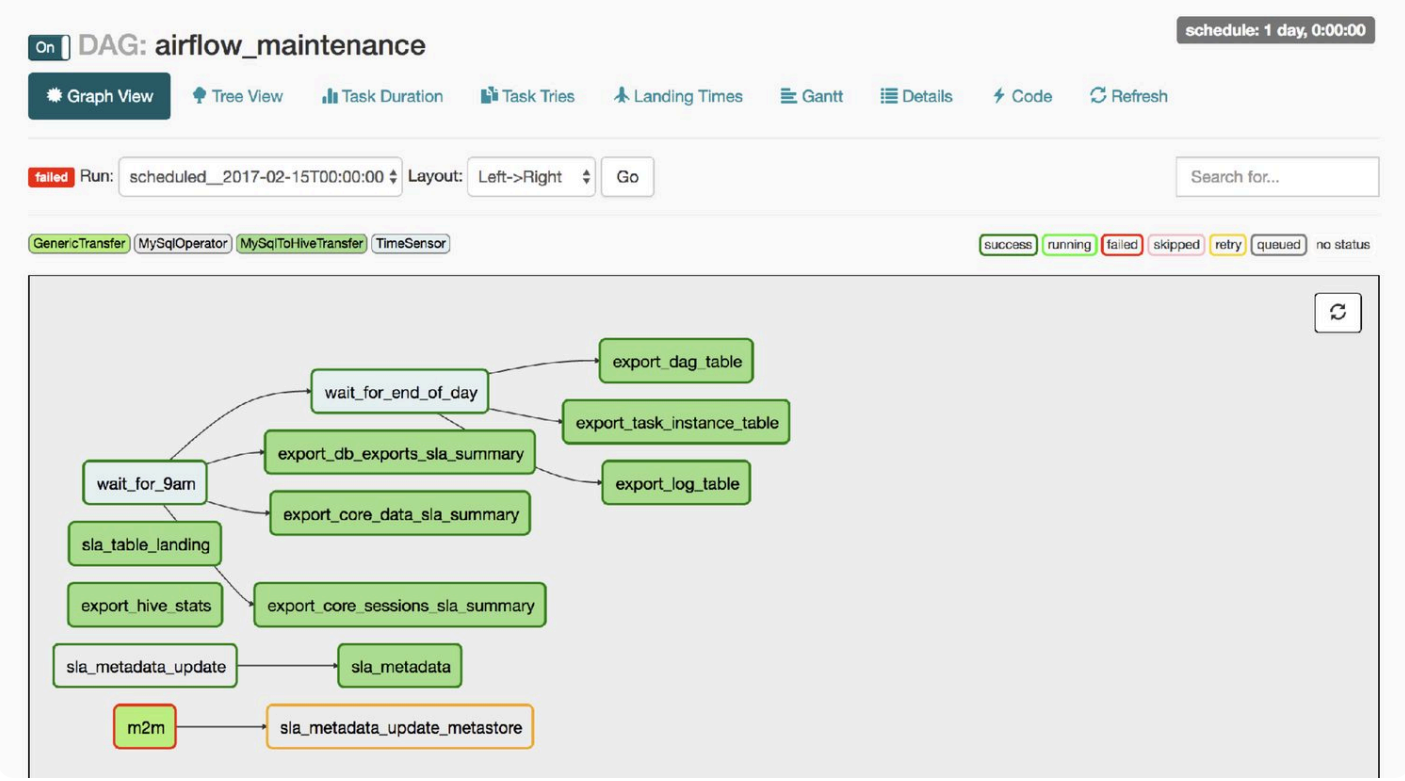
Historical architecture: focus on the responsibilities, rather than deployment details



Scheduling, task execution and recorded state are distinct responsibilities

Read a Real Dependency Graph

Historical Airflow maintenance DAG



Independent branches can progress while descendants of a failed task remain blocked

Demo: Build and Run a DAG

Predict runtime and verify the data, rather than reading task colors alone

Start run | Next event | Fix input and retry | Reset run | ☐ Product load fails | ☐ Edit dependencies | Workers 2

☐ Duplicate P1 in products



Run history · simulated minutes · t = 0

Load orders
Load products
Join inputs
Daily totals
Check data
Publish

Candidate · not published

No result yet

Two independent loads; join waits for both

Teaching simulation: 1–4 workers, fixed task durations; not an Airflow server

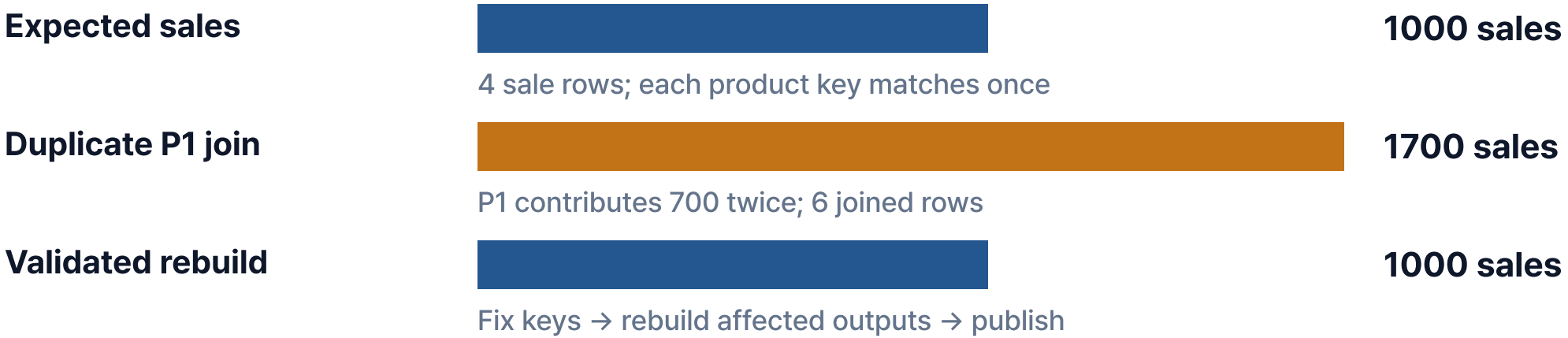
Answer: More Workers Have a Limit

Execution choice	Time	Limiting path
2 workers; independent loads	9	max(2,3) + 2 + 2 + 1 + 1
4 workers; same graph	9	The dependent chain is unchanged
1 worker, or serialize the loads	11	2 + 3 + 2 + 2 + 1 + 1

```
Lower bound = longest dependency path = 9 simulated minutes
```

Parallelism helps independent work; extra workers cannot shorten an unchanged dependent chain

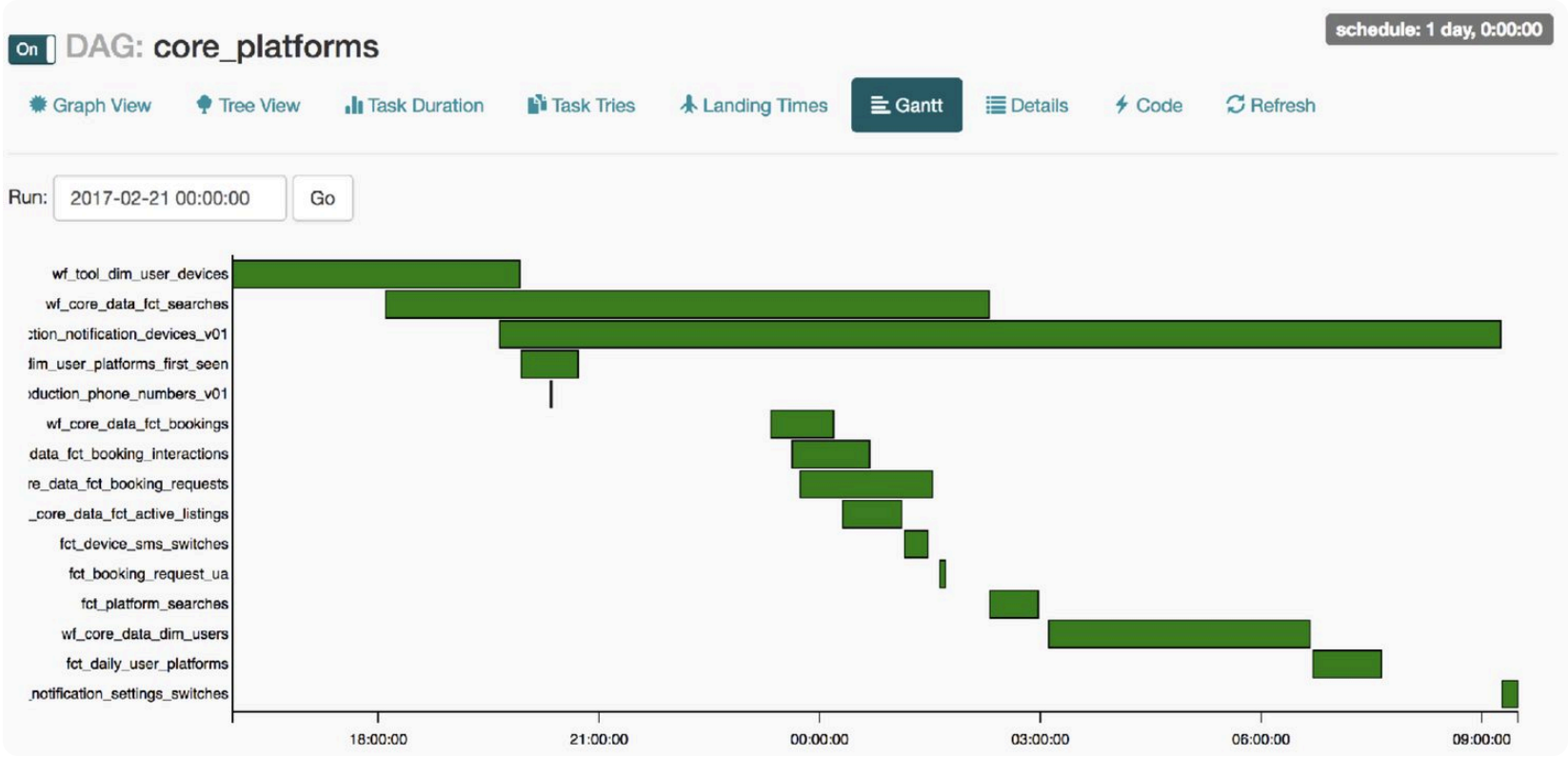
Answer: Successful Loads Can Produce Wrong Totals



Task success is not a data contract; a duplicated join can still return successfully

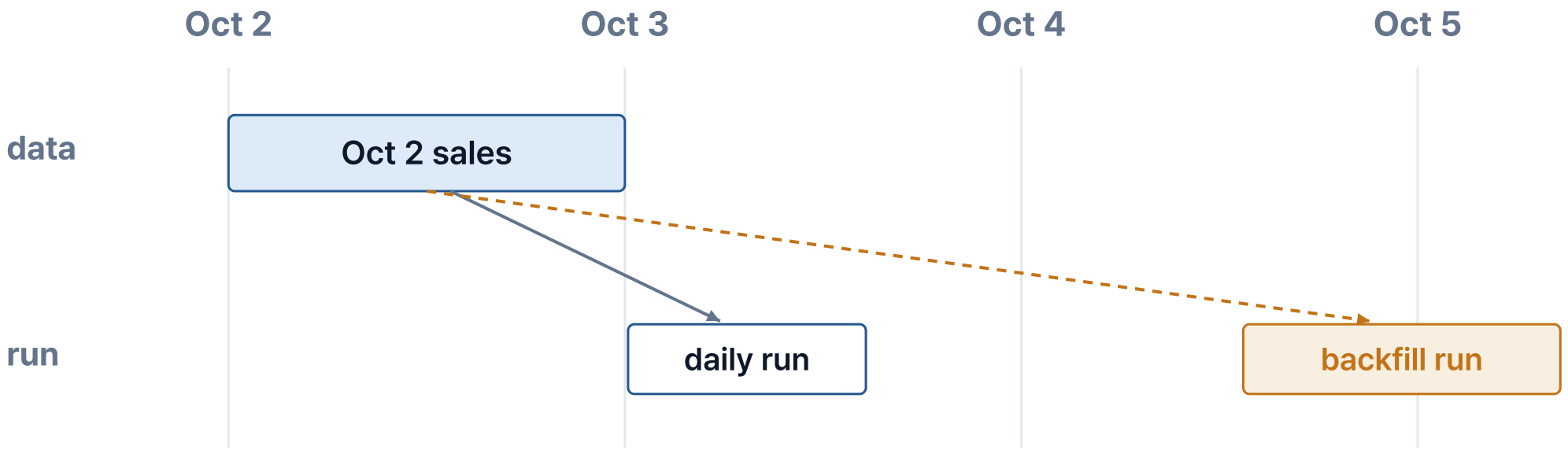
Read Execution Time in Gantt

Historical Airflow Gantt: actual overlap and long-running tasks



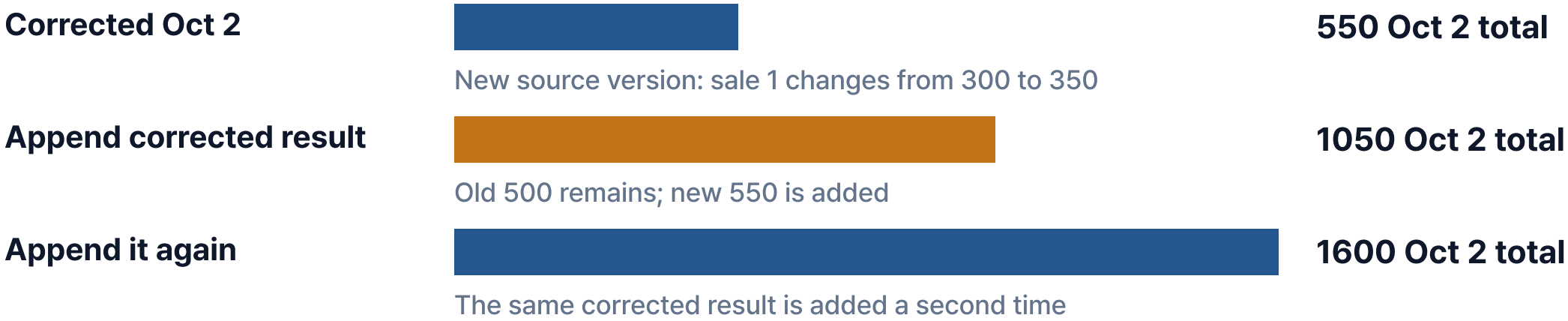
Use task durations together with dependencies to locate the path delaying completion

Data Interval and Run Time



The date being recomputed can differ from the time when the task runs

Caveat: Retrying an Append



The same corrected interval is counted repeatedly when append leaves the old result in place

Answer: Replace a Fixed Data Interval

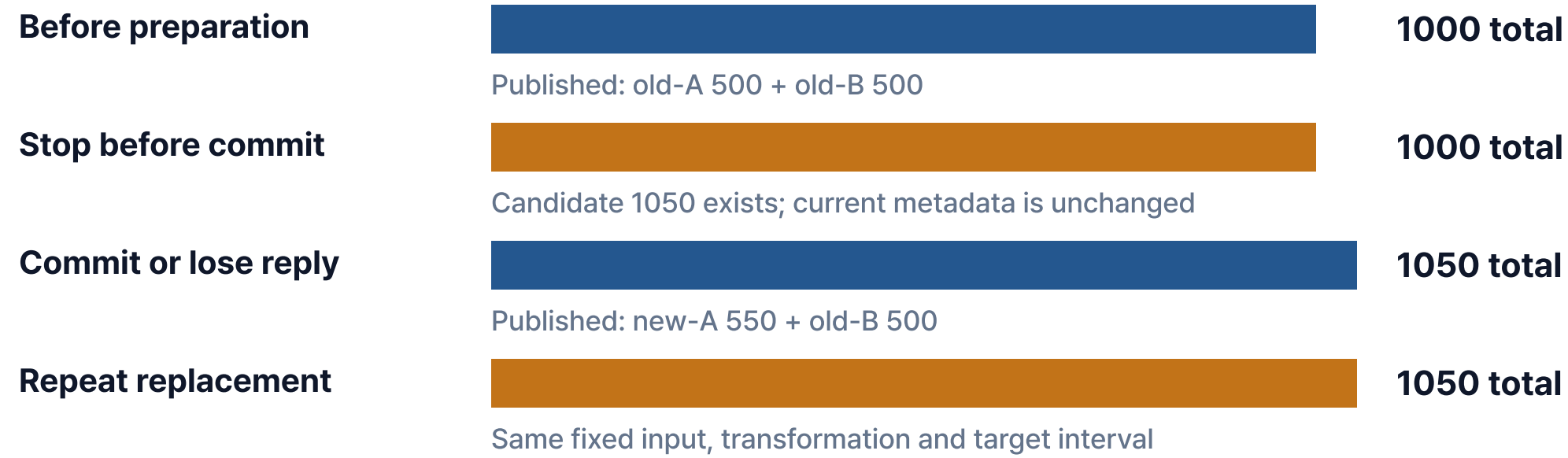
Fix the input version, transformation and target interval before rerunning

Target Oct 2 representation	Initial	First corrected run	Same run again
Append report rows	500	500 + 550 = 1050	500 + 550 + 550 = 1600
Replace Oct 2 result	500	550	550
Oct 3, outside target interval	500	500	500

The corrected interval is 550 after either one execution or repeated executions

A Replacement Must Survive Interruption

Commit the prepared interval; preserve the other interval



Prepare and validate first; an atomic publication prevents an empty or mixed replacement

Which Guarantee Failed?

Observed result	First hypothesis	Evidence to inspect
Correct number, excessive input read	Projection or partition pruning missing	Required columns and scanned date ranges
Complete published table totals 1700	Duplicate join matches	Product key uniqueness and joined row count
A retry changes yesterday again	Input or target interval was not fixed	Input version, interval and publication mode

Diagnose the observed failure before changing the tool or adding more workers

Takeaway: Read, Merge and Publish Correctly

- Read the required columns and partitions; preserve sufficient aggregation state
- Separate transformation location, retained grain and atomic table publication
- Validate outputs, then retry a fixed interval through a safe replacement